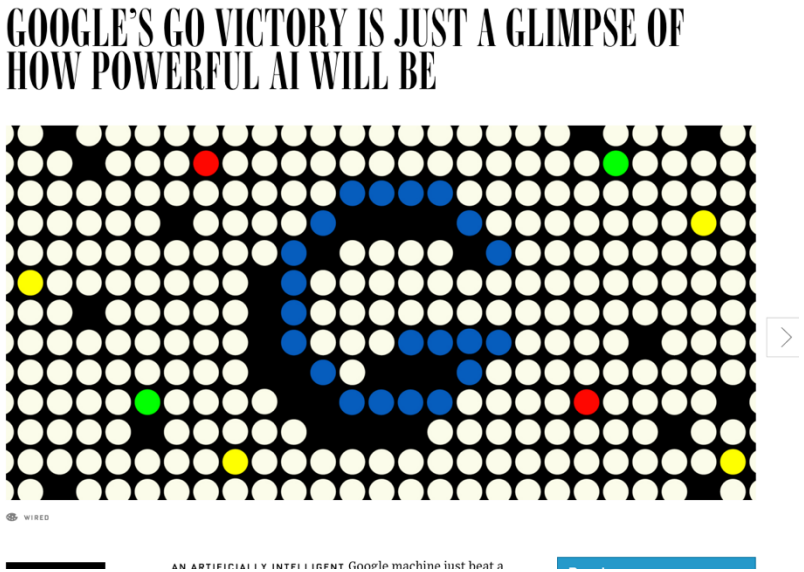


Adversarial Search

CHAPTER 5

Types of Games

	Deterministic	**Stochastic**
Perfect Information	chess, checkers, go	backgammon, monopoly
Imperfect Information	blind tictactoe	bridge, poker, scrabble, nuclear war

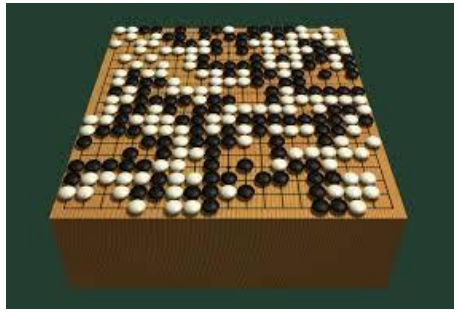
Deterministic Single Player Game?

- Deterministic Single Player, Perfect Information
  - Know the rules
  - Know the set of actions
  - Know when you win
  - Eg: Freecell, Rubik's cube, n-puzzle
- … it's just Search!

Adversarial Search

- "Unpredictable" Opponent -solution is a strategy
  - Specifying a move for every possible opponent move
- AI- Games – Zero Sum Games
  - Deterministic – Two player
  - Turn taking
  - Fully Observable



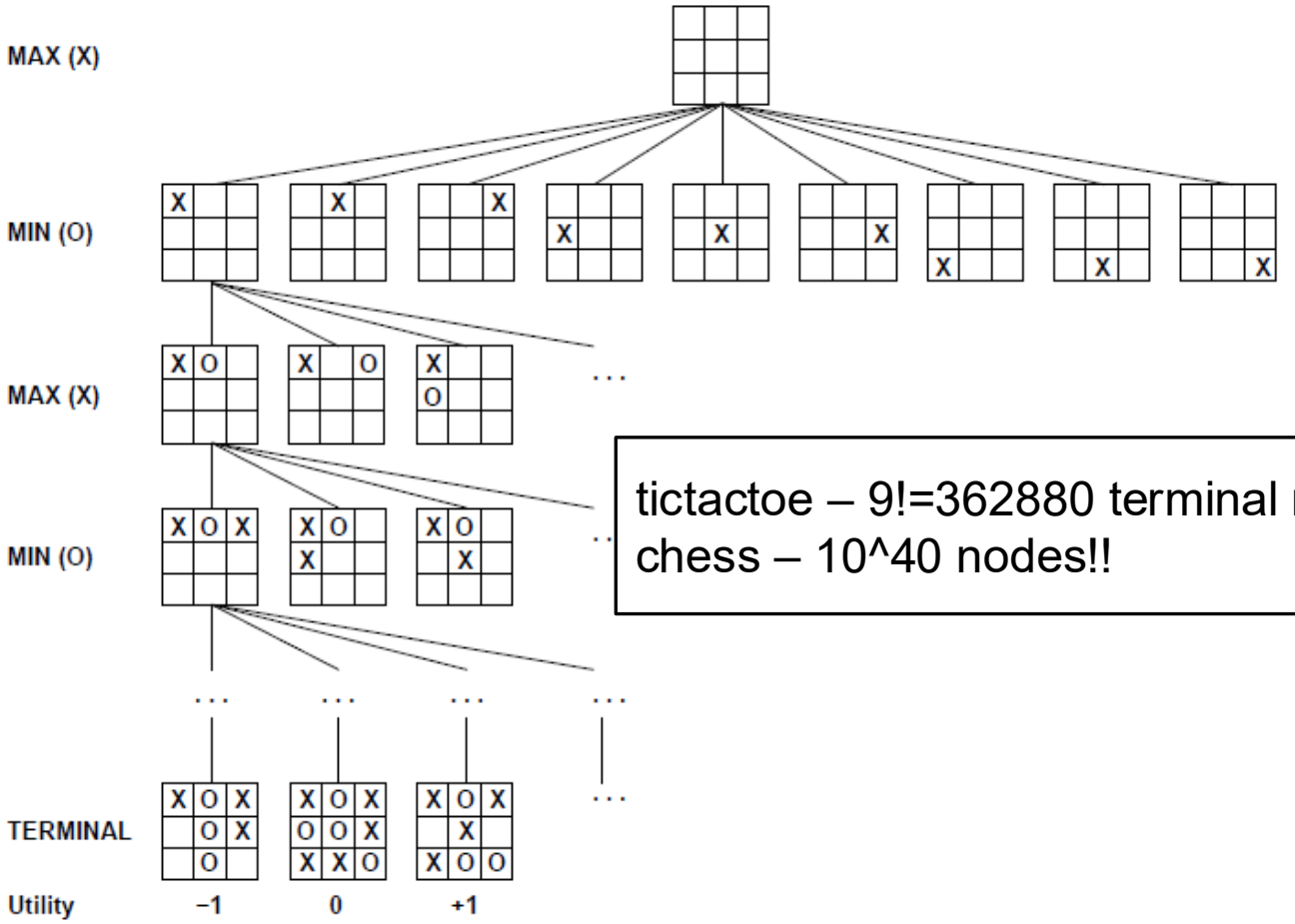


Deterministic Games

Formalization

- S_0 - initial state specifies initial setup of the game
- $Player(s)$: Defines which player has the move in a state
- $Actions(s)$: Returns the set of legal moves in a state
- $Result(s, a)$: Transition model, that defines the outcome of a move in a state
- $Terminal - Test(s)$: A terminal test, which is true if the game is over and false otherwise. States where the game has ended are called terminal states.
- $Utility(p, s)$: A utility/objective/payoff function that determines the final numeric value for a game that ends in the terminal state s for player p.

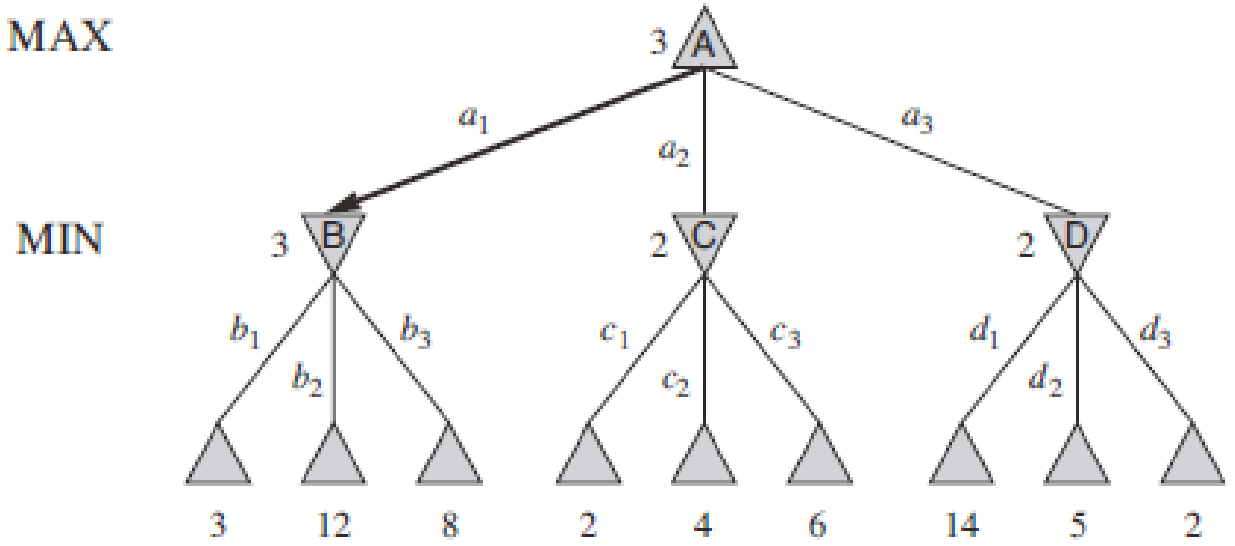
Game tree (2-player, deterministic, turns)



tictactoe – 9!=362880 terminal nodes
chess – 10^40 nodes!!

Minimax Algorithm

- Optimal decision in deterministic, perfect information games
- Idea : choose the move resulting in the highest minimax value
- Ex: 2ply game



Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action
    inputs: state, current state in game
    return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))
```

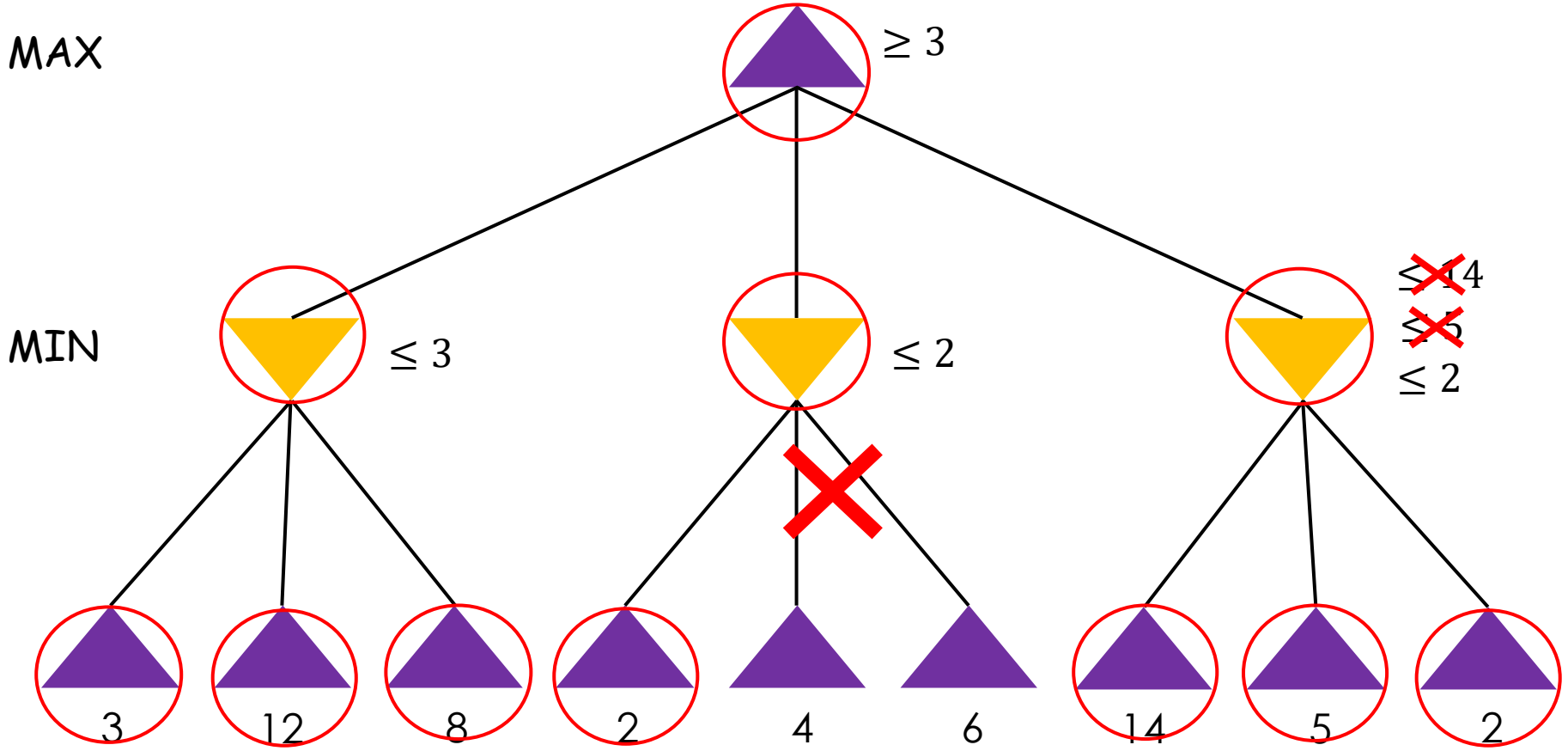
```
function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do v ← MAX(v, MIN-VALUE(s))
    return v
```

```
function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do v ← MIN(v, MAX-VALUE(s))
    return v
```

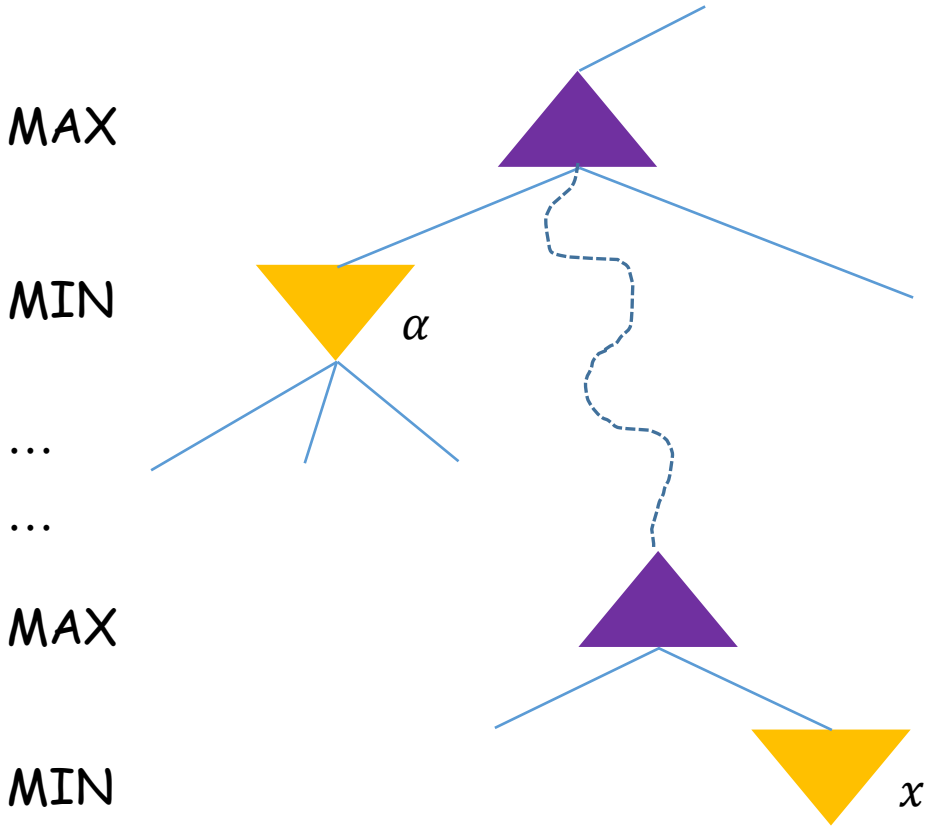
Minimax Algorithm: Analysis

- Completeness: Yes if the tree is finite
- Optimality: Yes, against an optimal opponent. Otherwise??
- Time Complexity: $O(b^m)$
- Space Complexity: $O(bm)$ – depth first exploration
- For chess - $b \approx 35, m \approx 100$, for 'reasonable' games
  - Exact solution is infeasible
- But do we need to explore every path?

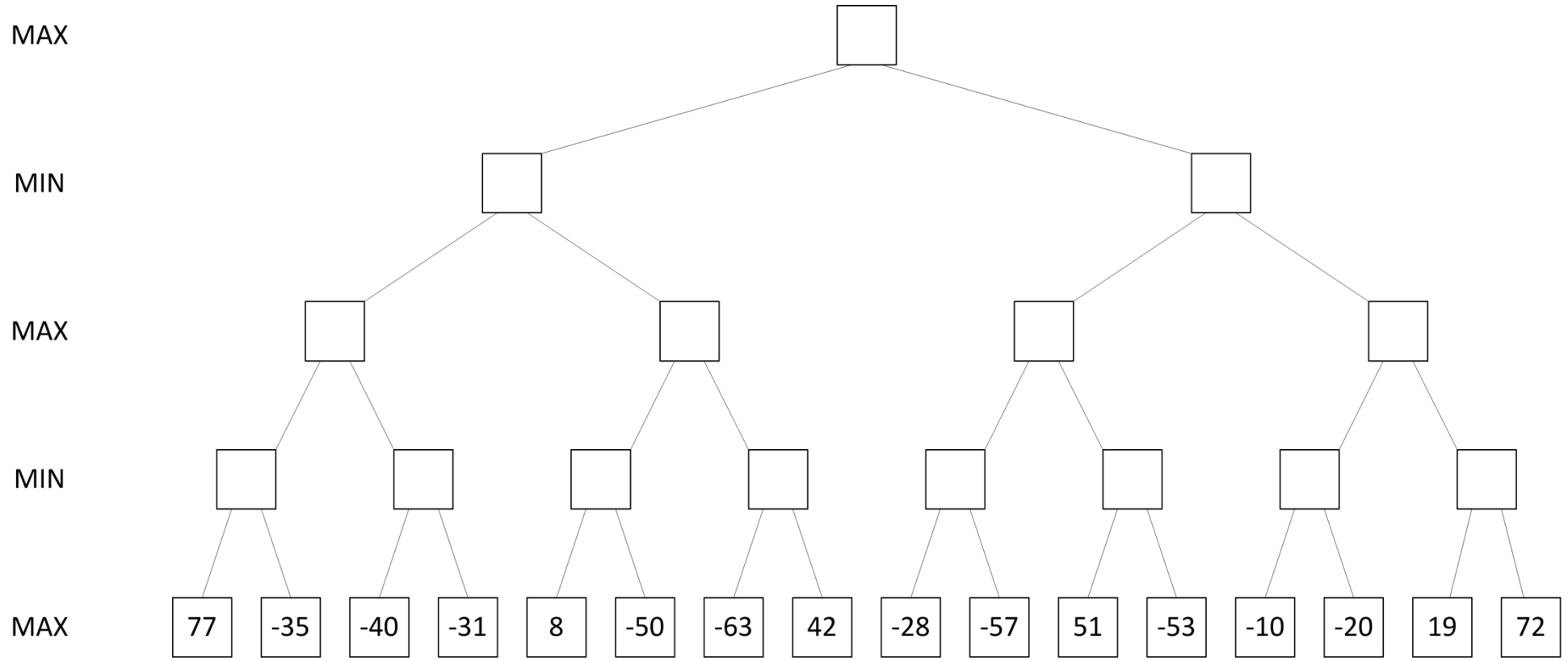
$\alpha - \beta$ Pruning Example

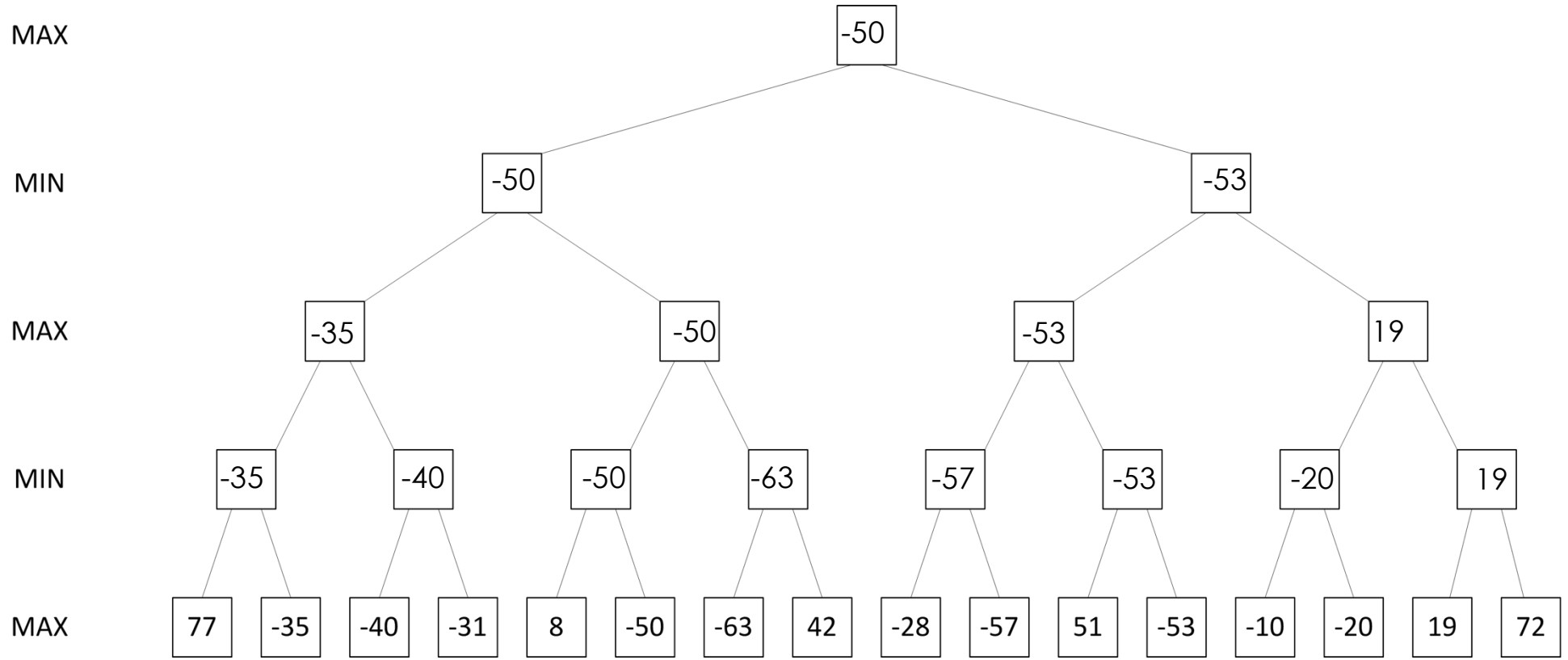


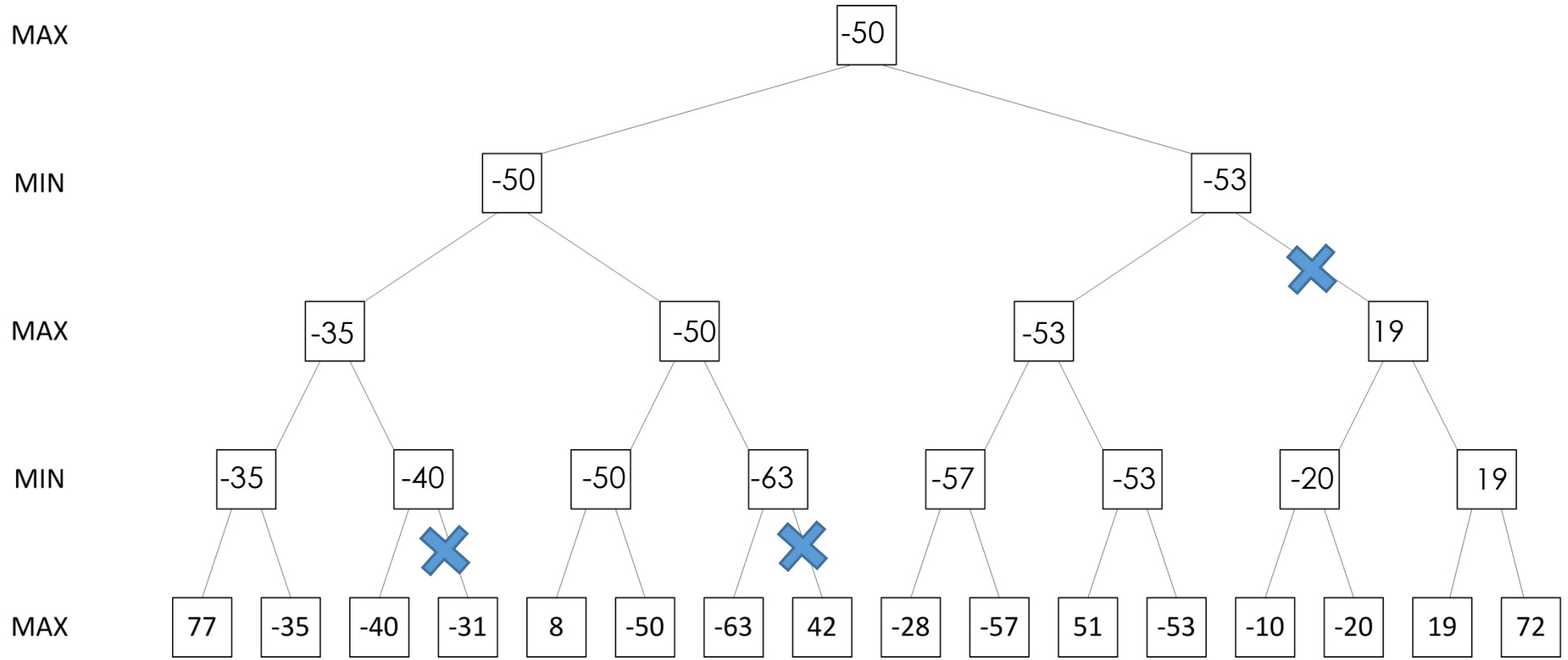
$\alpha - \beta$ Pruning



- α – the highest value we have found so far at any choice point along the path for MAX
- If $x < \alpha$, MAX will avoid it → prune the branch
- Define β similarly for MIN


MAX
MIN
MAX
MIN
MAX
77
-35
-40
-31
8
-50
-63
42
-28
-57
51
-53
-10
-20
19
72

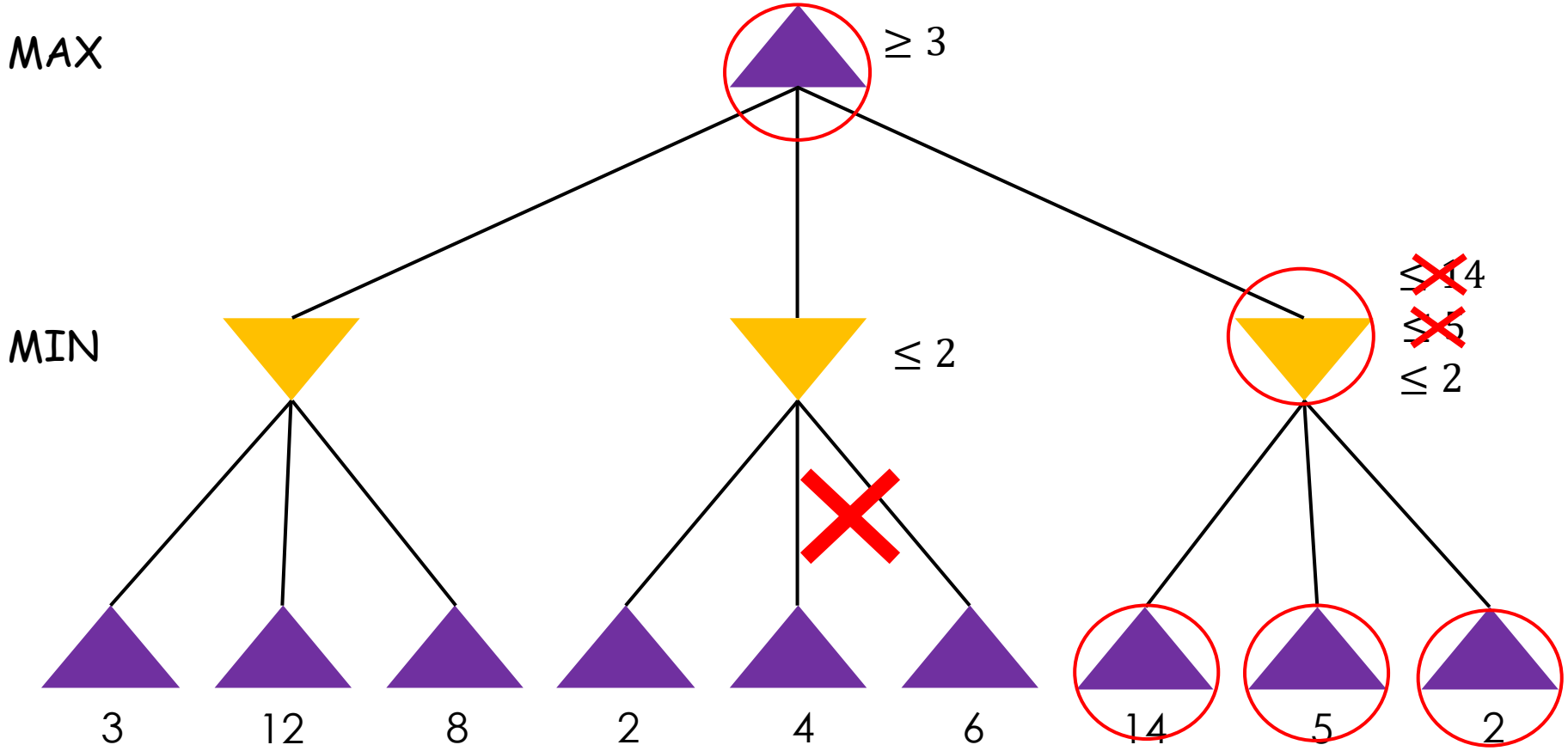

MAX
MIN
MAX
MIN
MAX
-50
-50
-53
-35
-50
-53
19
-35
-40
-50
-63
-57
-53
-20
19
77
-35
-40
-31
8
-50
-63
42
-28
-57
51
-53
-10
-20
19
72


MAX
MIN
MAX
MIN
MAX
-50
-50
-53
-35
-50
-53
19
-35
-40
-50
-63
-57
-53
-20
19
77
-35
-40
-31
8
-50
-63
42
-28
-57
51
-53
-10
-20
19
72

$\alpha - \beta$ Pruning - Analysis

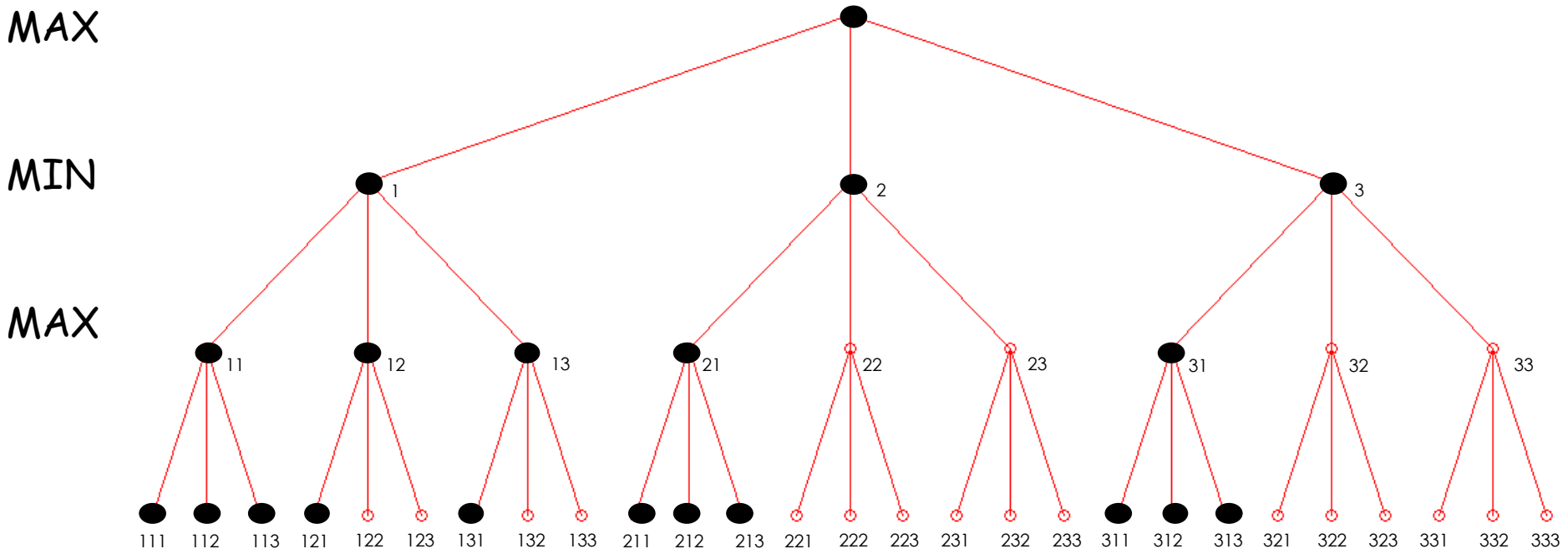
- Pruning does not affect the final result.
- Does ordering effect the pruning process?

Order effect on Pruning



Order effect on Pruning

❑Best Case Scenario



- ❑ In the best case the time complexity reduced to $O\left(b^{\frac{m}{2}}\right)$
- ❑ Or in the same time the $\alpha - \beta$ pruning approach can search a tree of double the depth $(2m)$!

$\alpha - \beta$ Pruning - Analysis

- Pruning does not affect the final result.
- Does ordering effect the pruning process?
  - Best case $O(b^{\frac{m}{2}})$
  - Random (instead of best first search) - $O(b^{\frac{3m}{4}})$
- Iterative deepening
  - Change the ordering while doing the search
- Using information from past explorations
  - Transposition tables
- 35^{50} is still a very large search space

Imperfect Real-Time Decisions

- Claude Shannon: programming a computer to play chess
  - Use EVAL (evaluation function) instead of UTILITY
  - Use CUTOFF-TEST instead of TERMINAL-TEST

Evaluation Functions

- Estimate of the expected utility of the game from a given position
- Performance depends strongly on the quality of the function
- EVAL should order the terminal states in the same way as the utility function
- Computation must not take too long
- For non-terminal states, EVAL should strongly correlate to the actual chances of winning

$$EVAL(s) = \sum_{i=1}^{N} w_i f_i(s)$$

Cutting Off Search

- Deeper is better but why?

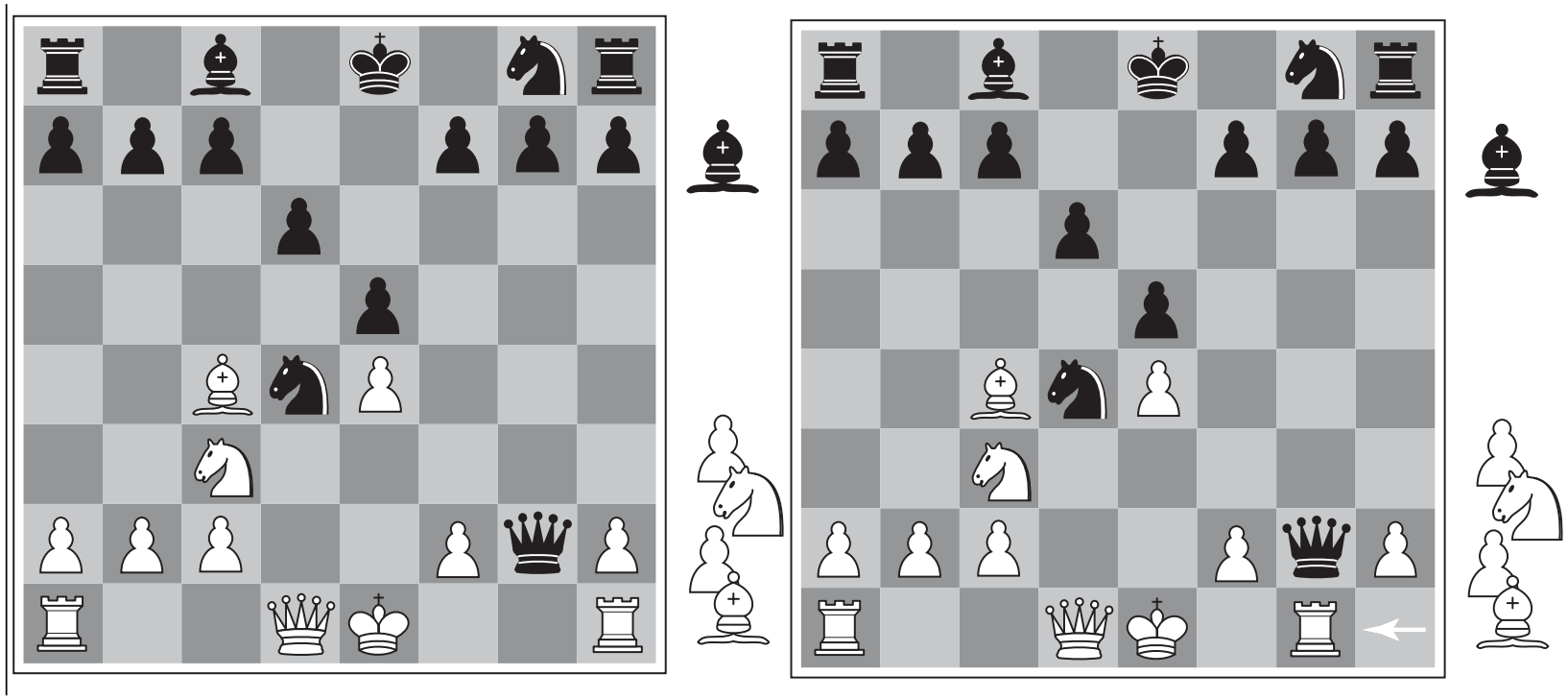
Deeper is Better

- Possible reasons
  - Propagation of evaluation values through mins and maxs improves the collective accuracy?
  - Going deeper makes the agent look for “traps”, thus improving the evaluation function?

Cutting Off Search

- Deeper is better but why?
- At what depth to cutoff search?
  - A move is selected within the allocated time
  - Iterative deepening?
- Uniform Depth
  - Quiescence search – go deeper when the game state is in a flux

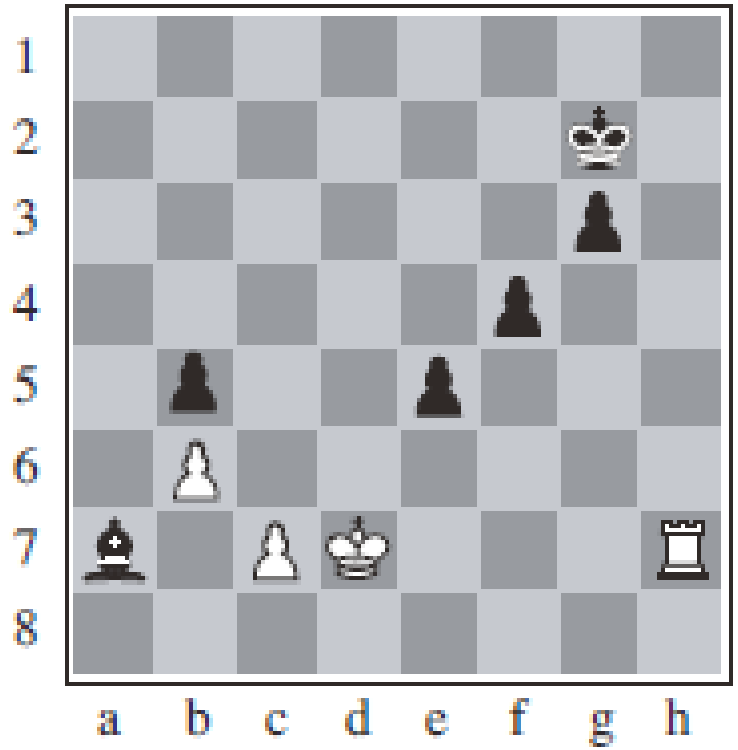
Quiescence Search



Cutting Off Search

- Deeper is better but why?
- At what depth to cutoff search?
  - A move is selected within the allocated time
  - Iterative deepening?
- Uniform Depth
  - Quiescence search – go deeper when the game state is in a flux
- Horizon Effect
  - Serious damage that is temporarily avoided by delaying tactics

Horizon effect



Can the black bishop be saved?

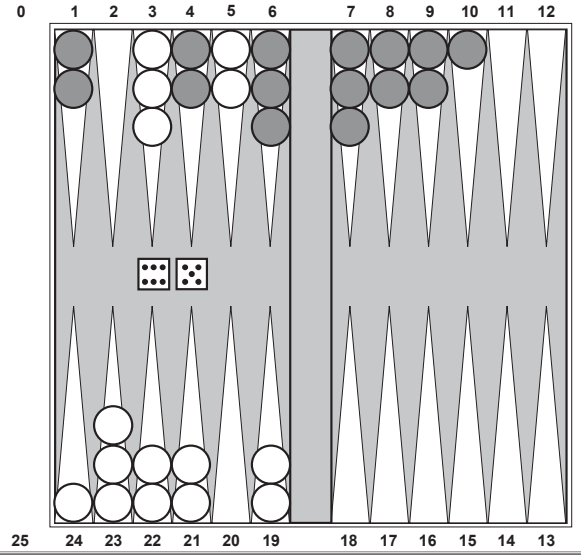
Imperfect Real-Time Decisions

- Claude Shannon: programming a computer to play chess
  - Use EVAL (evaluation function) instead of UTILITY
  - Use CUTOFF-TEST instead of TERMINAL-TEST
- Forward Pruning
  - Beam search at each ply
  - Probabilistic Cut Algorithm

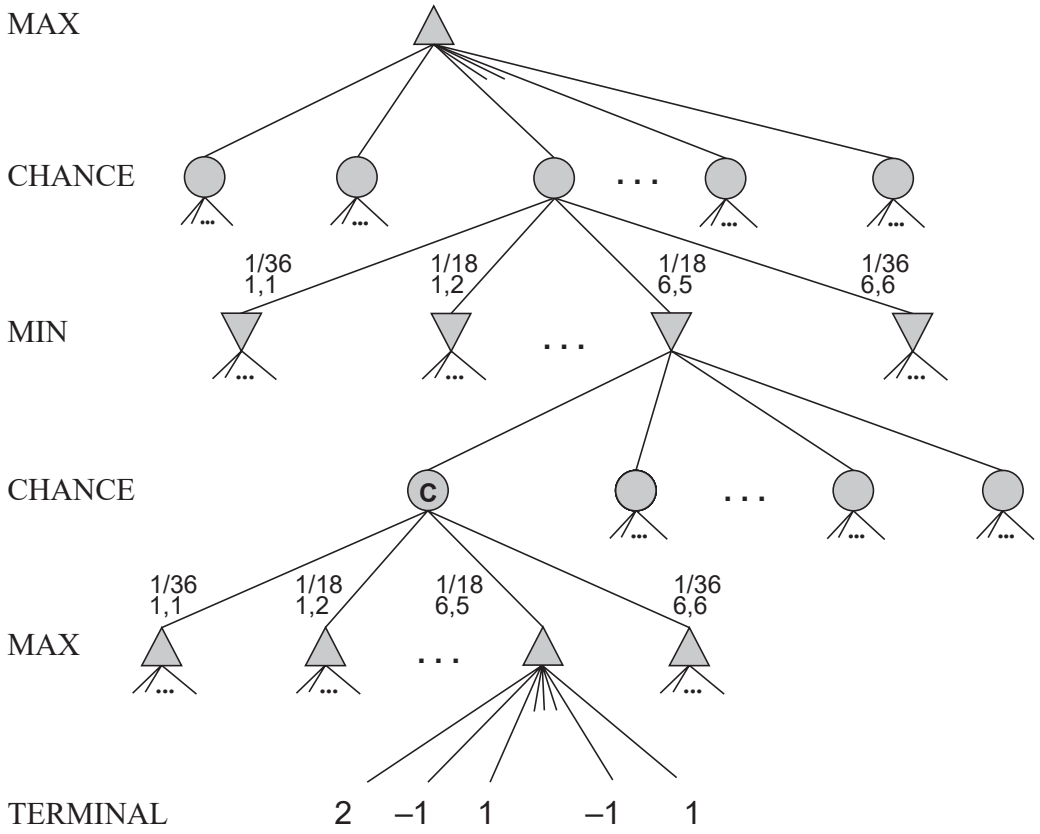
Fun Trivia
- 4ply ~ human novice chess player
- 8ply ~ typical PC, average human chess player
- 12ply ~ Deep Blue, Grand Masters

Stochastic Games

- Games that include an element of randomness
  - Throw of dice
  - Eg. Backgammon
- Game tree includes chance nodes
  - Edges out of chance nodes indicate the possibilities with their probability of occurrences

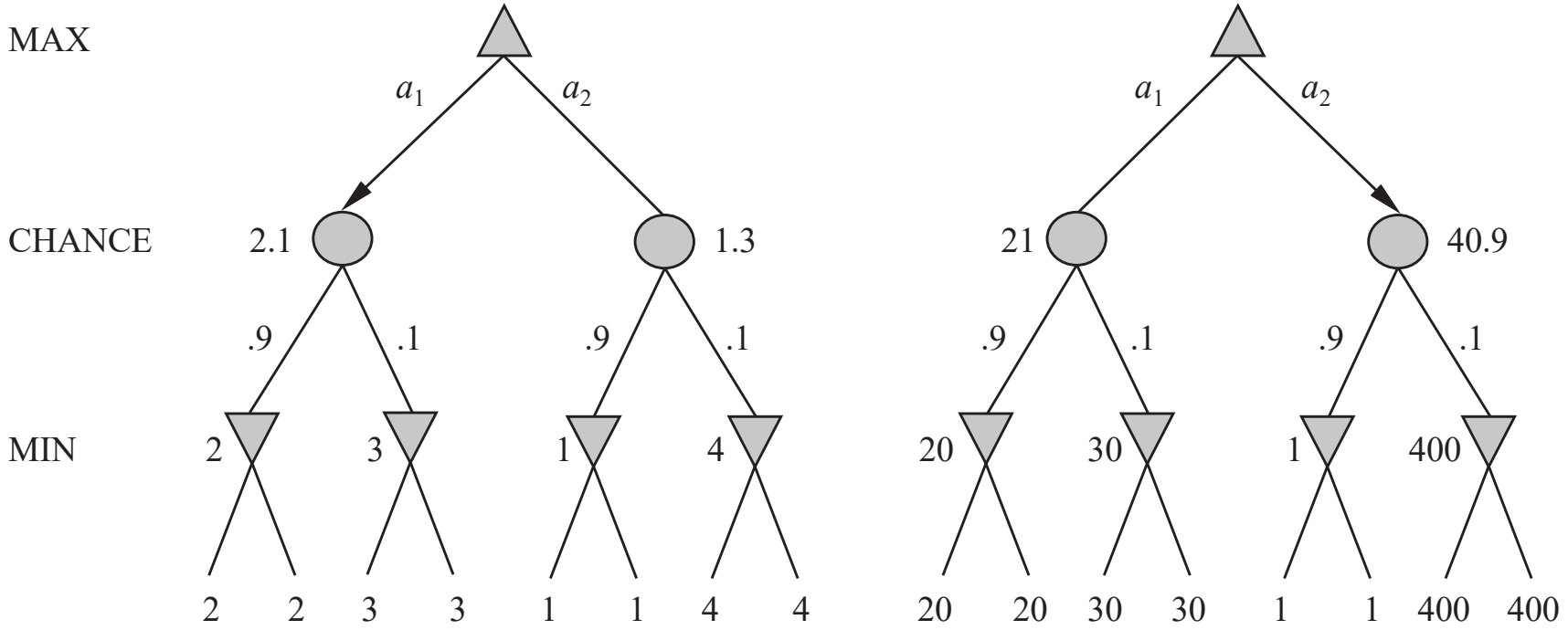


Expectiminimax



$$\text{EXPECTIMINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \\ \sum_r P(r)\,\text{EXPECTIMINIMAX}(\text{RESULT}(s,r)) & \text{if PLAYER}(s) = \text{CHANCE} \end{cases}$$

Evaluation Functions for Stochastic Games



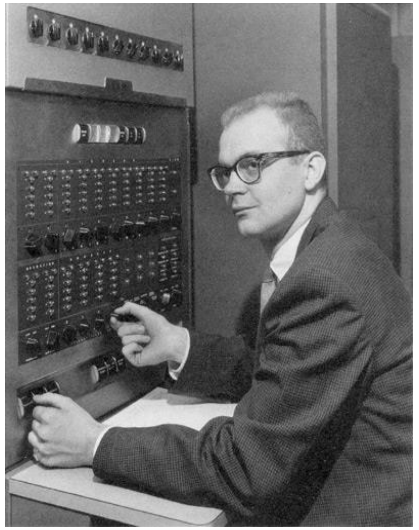
Scaling evaluation values changes the action path

Expectiminimax - Analysis

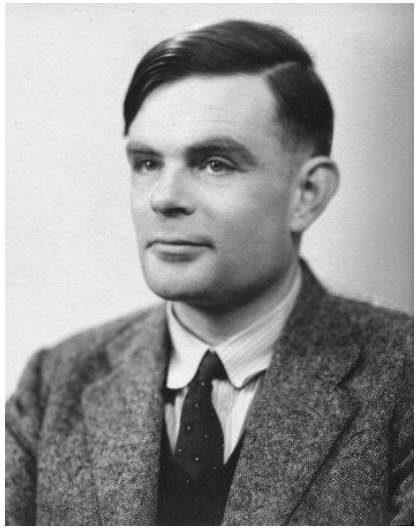
- Include n- chance nodes
  - $O(b^m n^m)$ – assuming uniform branching
- Extra cost associated with chance nodes makes it unrealistic for looking ahead very far.
- Can alpha-beta pruning be applied?
  - Yes – one way to do it is make bounds on the values of the utility function
  - Alternative – Monte-Carlo Simulation to evaluate a position (explore later)

John Von
Neumann
Min-max theorem

John McCarthy
Alpha-Beta Pruning



Donald Knuth
Alpha-Beta
Analysis



D. G. Prinz, Alan Turing
Chess Program

IBM Deep Blue